Predicting Electronic Structures at Any Length Scale with Machine Learning

B24309- DUSHANT SAKHARKAR, V24033- KHUSHI BATRA

Based on: Fiedler et al., npj Computational Materials (2023) 9:115

27/04/2026

ROADMAP — Presentation Outline

1. **01 Introduction & Motivation:** Why do we need a DFT alternative?
2. **02 DFT & Its Limitations:** Cubic scaling bottleneck
3. **03 MALA Framework:** How ML replaces DFT
4. **04 Bispectrum Descriptors:** Encoding local atomic environments
5. **05 Radial Distribution Function:** Validating size transferability
6. **06 Training & Inference:** The notebook workflow
7. **07 Scaling & Results:** Speed and accuracy benchmarks
8. **08 Future Extensions:** Fermi energy, band structure & beyond

INTRODUCTION — Why Electronic Structure Matters

Electrons govern virtually all material properties — from batteries to semiconductors to planetary interiors.

- **Energy Storage:** Better batteries & fuel cells require understanding electron transport and reactivity at interfaces.
- **Catalysis:** ML-accelerated screening of catalytic reactions to find more efficient CO_2 reduction pathways.
- **Semiconductors:** Device-scale modeling of electron densities in transistors, solar cells, and LEDs.
- **Planetary Science:** Electronic structure governs energy transport inside giant gas planets under extreme conditions.

BACKGROUND — Density Functional Theory (DFT): Power & Limits

What DFT Does Well

- ✓ Nobel Prize-winning framework (Kohn 1998)
- ✓ Quantum-mechanical accuracy for ∼100–1000 atoms
- ✓ Determines electron density, energies, forces
- ✓ Key to battery, catalyst & materials research
- ✓ Widely implemented: Quantum ESPRESSO, VASP

Critical Limitations

- × Scales as $\mathcal{O}(N^3)$ — cubic with system size
- × Routine limit: only a few hundred atoms
- × ∼1000 atoms → intractable on most HPC
- × Cannot study stacking faults at real device scale
- × Finite-size errors in large-scale properties

DFT complexity: $\mathcal{O}(N^3) \rightarrow$ **MALA:** $\mathcal{O}(N) \rightarrow$ **up to** $1000\times$ **speedup**

MALA — The MALA Framework: How ML Replaces DFT

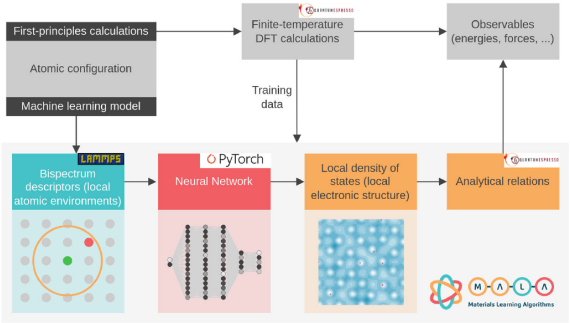
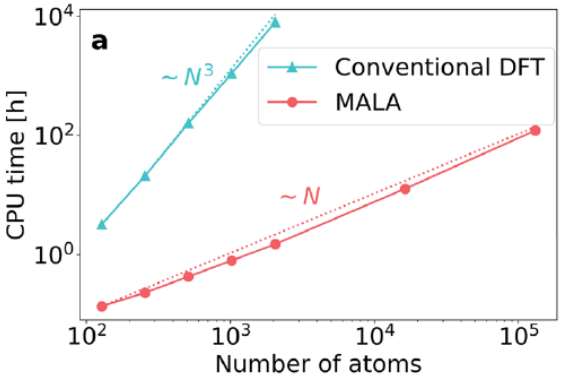


Fig. 1 Overview of the MALA framework. ML models created via this workflow can be trained on data from popular first-principles simulation codes such as Quantum ESPRESSO[42]. The pictograms below the individual workflow steps show, from left to right, the calculation of local descriptors at an arbitrary grid point (green) based on information at adjunct grid points (gray) within a certain cutoff radius (orange), with an atom shown in red; a neural network; the electronic structure, exemplified here as a contour plot of the electronic density for a cell containing Aluminum atoms (red).

Key Insight

The NN is trained locally — it has no awareness of system size. Linear scaling follows naturally.

SCALING — From $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$: Breaking the Bottleneck



Summary: N^3 DFT Scaling vs N MALA Scaling $\rightarrow \sim 1000\times$ Max Speedup

DESCRIPTORS — Bispectrum Descriptors: Encoding Local Environments

Atomic Density: $\rho(r) = \delta(0) + \sum f_c(|r_k|, R_{\text{cut}}) \cdot w_{\nu k} \cdot \delta(r_k)$
Expanded into 4D hyperspherical harmonics $\rightarrow$ yields descriptor vector $B(J, r)$

- **Cutoff Radius R_{cut}:** Controls how far away atoms are included in each descriptor. Too small $\rightarrow$ misses non-local effects. Too large $\rightarrow$ encodes entire cell, task becomes too complex. For Beryllium: $R_{\text{cut}} = 4.67637$ Å
- **Feature Dimension J_{max}:** Controls number of hyperspherical harmonics used. Too small $\rightarrow$ descriptor misses atomic density features. Too large $\rightarrow$ introduces noise, complicates training. For Beryllium: $J_{\text{max}} = 10$

Why Bispectrum Descriptors Enable Linear Scaling

- Each grid point's descriptor depends only on nearby atoms within R_{cut} — independently evaluated by LAMMPS.
- There is no global matrix. The NN maps each descriptor independently $\rightarrow$ natural $\mathcal{O}(N)$ workflow. Non-locality is captured within R_{cut} without coupling grid points globally.

RDF — What It Is & Why MALA Needs It

Definition: $g(r) = \frac{1}{\rho N V(r)} \sum \sum \delta(r - |r_i - r_j|)$

Average ion density in a shell $[r, r + dr]$ around a reference atom, relative to an ideal isotropic system of density $\rho = N/V$

Physical Interpretation

- $g(r) = 1 \rightarrow$ ideal gas (random distribution)
- $g(r) > 1 \rightarrow$ more likely to find an atom at r
- $g(r) = 0 \rightarrow$ atom cannot exist at r (exclusion zone)
- Sharp peaks $\rightarrow$ crystalline / ordered phases
- Broad peaks $\rightarrow$ liquid or amorphous phases
- Converges to 1 at large r (bulk-like behavior)

Why MALA Uses RDF

- Bispectrum input $B(J, r)$ depends on atoms within R_{cut}
- If RDFs match up to $R_{\text{cut}} \rightarrow$ same local environments $\rightarrow$ same input vectors $B(J, r)$
- Same inputs $\rightarrow$ NN interpolates, not extrapolates
- Validates large-cell predictions (e.g. comparing $N = 256$ vs $N = 100,000$)

RDF — NOTEBOOK — RDF Calculation: Project Notebook Implementation

```
from mala.targets import Target
from ase.io import read
# Load atoms & set unit cell
atoms = read('Be128_snapshot0.xyz')
atoms.set_cell(cell)
atoms.set_pbc(True)  # PBC

# Calculate RDF
rdf, radii = Target.radial_distribution_function_from_atoms(
    atoms, bins=500, rMax='mic'


# Compare N=256 vs N=2048
plt.axvline(x=4.7, linestyle=':')
# ^ Bispectrum cutoff radius
```

rMax='mic': Uses Minimum Image Convention — half the cell edge length. Ensures $g(r)$ is well-defined.

RDF OUTPUT

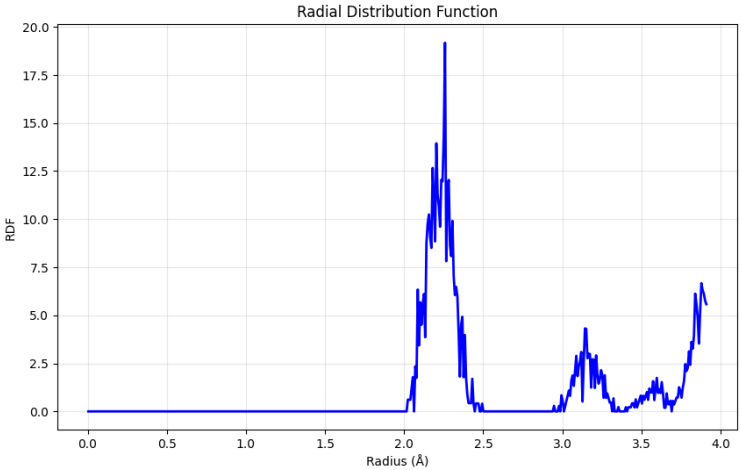


Figure: RDF at N = 256 Atoms

TRAINING — Neural Network Training: The LDOS Surrogate

Network Architecture (Beryllium)

- **Input layer:** Bispectrum descriptors $B(J, r)$
- **Hidden layer 1:** 400 neurons — LeakyReLU
- **Hidden layer 2:** 400 neurons — LeakyReLU
- **Output layer:** LDOS vector $d(\epsilon, r)$ — 250 points

Training Data Pipeline

1. DFT-MD snapshots at 298 K
2. Quantum ESPRESSO $\rightarrow$ LDOS data (k-grid: $12 \times 6 \times 6$ for 256 atoms)
3. Snapshots 0,1,3,4 $\rightarrow$ training
4. Snapshots 2,20 $\rightarrow$ validation
5. Save: `.network.pth`, `.iscaler.pkl`, `.oscaler.pkl`, `.params.json`

Key Training Hyperparameters

- Optimizer: Adam — LR: 0.0001 — Batch size: 64
- Max epochs: 10,000 — Early stopping: 30 epochs
- Input rescaling: feature-wise standard — GPU: enabled

INFERENCE — Model Inference: Why & How We Run It

Why Inference?

Inference applies a trained model to new atomic structures — sizes never seen during training. This is the key test of generalization: can a model trained on 256 atoms predict the electronic structure of 100,000 + atoms accurately?

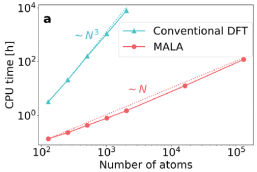
1. **Load & Configure**
   Load trained model weights (.pth), scalers (.pkl), and parameters (.json). Set descriptor type (Bispectrum, J=10), LDOS grid, and MS-MPI + LAMMPS paths for descriptor computation.
2. **Compute Descriptors**
   LAMMPS calculates bispectrum descriptors $B(J, r)$ for each real-space grid point of the new (larger) atomic structure. This is the computationally light $\mathcal{O}(N)$ step.
3. **Predict & Post-Process**
   NN maps each $B(J, r) \rightarrow$ LDOS $d(\epsilon, r)$. Integrate LDOS to get: electron density $n(r)$, total free energy A, density of states $D(\epsilon)$. Save `density.npy`.

RESULTS — Accuracy & Scaling Benchmarks

< 43 meV atom^{-1}
Total Energy Error
(chemical accuracy)

121 h CPU
131,072-atom run
(vs 7920 h for DFT)

Atoms	DFT CPU-hrs	Onboard CPU-hrs	Speedup
256	21.0	0.23	$\sim 91\times$
1024	1075	0.80	$\sim 1344\times$
2048	7920	1.50	$\sim 5280\times$
7,384	N/A	12.78	∞
100,000	N/A	~ 100	∞



TRANSFERABILITY — Size Transferability: Training Small, Predicting Large

Core Principle: Because each NN evaluation is local (one grid point at a time), a network trained on 256-atom cells is performing interpolation — not extrapolation — when applied to 100,000+ atom systems, provided local environments match.

- **Train on N=256:** Model learns LDOS from small, tractable DFT cells. Only 2 snapshots needed for a transferable model.
- **RDF Verification:** Compare $g(r)$ for training (N=256) vs prediction (N=100,000+ atoms) cells up to R_{cut}. Matching curves $\rightarrow$ same local environments $\rightarrow$ interpolation.
- **Infer on N= 100,000+ atoms :** Model predicts electronic density and total energy with errors equivalent to the benchmark scale — verified by radial distribution analysis.

Stacking Fault Demonstration: Model correctly reflects energy differences between hcp and fcc Beryllium cells following expected $N^{-1/3}$ scaling — without any retraining.

Conclusions

- MALA uses a feed-forward NN to learn LDOS locally, breaking DFT's cubic scaling curse entirely.
- Achieves up to 1000$\times$ speedup, enabling 131,072-atom calculations that are infeasible with DFT.
- Maintains chemical accuracy ($<$ 43 meV/atom energy error, $<$ 1% density MAPE) at all scales.
- RDF analysis confirms size transferability — predictions are interpolations, not risky extrapolations.
- Open-source MALA framework integrates LAMMPS, PyTorch, and Quantum ESPRESSO end-to-end.

FUTURE WORK — Extending MALA: Any Trainable Atomic Target

The MALA framework is target-agnostic. Any quantity expressible per grid point or per atom can be the output layer.

- **Fermi Energy / Chemical Potential:** Train the output layer to predict $\mu(T)$ directly from local atomic environments. Enables fast Fermi level tracking in semiconductor doping studies.
- **Band Structure Prediction:** Output the band energies $\epsilon_j(k)$ at selected k-points. The NN can learn band topology of materials — useful for topological insulator screening.
- **Electron Density Directly:** Bypass LDOS entirely and train $n(r)$ as the target. Faster inference for charge-density-sensitive properties like work function or Bader charge analysis.
- **Atomic Forces / Stress Tensor:** Differentiate the predicted energy w.r.t. atomic positions to generate forces, replacing classical ML interatomic potentials with higher accuracy.

Key: Only the output layer and loss function change — descriptors, NN backbone, and $\mathcal{O}(N)$ workflow remain identical.

Thank You

Questions & Discussion

Fiedler et al. npj Computational Materials (2023) 9:115
MALA Code: https://doi.org/10.5281/zenodo.5557254
Training Data: https://doi.org/10.14278/rodare.1834